7

Chapter 7 – Case Study: Contributing to an open source project

If you made it this far you must be thrilled and wanting to get your hands dirty. Use this chapter as your first guided tour to contribute to a real project. All the knowledge you've gained thus far can be applied following the step by step example provided in this chapter.

The subject of our study is the DB2 module to Ruby on Rails. Why? You may ask. Because the authors of this book are familiar with it and it will give us the opportunity to take a look to a very important tool.

7.1 Ruby on Rails and the DB2 module

There are a few tools that can help you develop Web applications, Ruby on Rails (RoR) is certainly one of the best. Ruby on Rails is a Web application framework developed by David Hansson and released in 2004, since then it has gained a lot of traction and has become widely used. Its adoption is much due to its simplicity and philosophy. Ruby on Rails builds on principles such as, Don't Repeat Yourself and Convention over Configuration. RoR is based on the Model-View-Controller (MVC) architecture and built-in tools such as scaffolding, which guarantee a rapid and easy development.

DB2 is a relational database management system, first released in 1983 by IBM. Even though it is not an open source software, DB2 has a no-charge version known as DB2 Express-C, giving us the ability to use one of the most high-performance relational databases available in the market for free!. DB2 can run on Linux, UNIX, Windows, Mac OS X (beta version at the time of writing), system i and z/OS® (mainframes). Its high availability and seamless stability has catered to the business crowd, but fear not, its features may well be what you are looking for. Like Ruby on Rails, DB2 builds on bold principles to succeed, DB2 supports for pureXML® technology which gives it a clear boost on application performance. It also provides the best compression support in the industry, allowing huge databases to be reduced to slightly more than half of their previous sizes.

Bringing together the benefits of these two technologies will put you at an obvious advantage. Here is where the DB2 Ruby on Rails module comes in. The DB2 Ruby on Rails module is the glue that makes this happen. A team of six developers ensures the project stays highly active and extremely well maintained, providing Rails developers with continuous integration.

Note:

If you would like to learn more about Ruby on Rails, refer to the ebook *Getting started with Ruby on Rails*. For DB2, refer to the ebook *Getting started with DB2 Express-C*, or review *Appendix B, Up and Running with DB2*.

Both ebooks are free and part of the DB2 on Campus book series at ibm.com/db2/books.

7.2 The ruby forge

DB2 module for Ruby on Rails is hosted at ruby forge. A forge is basically a Web collaboration platform, providing developers with all the tools they need to manage the project and bring the discussion and code back to the community. A good forge will provide services like, mailing lists, source code management, wiki, bug tracking, forums and download archives. A forge holds a different number of projects, and some forges even have the ability to fork projects, enabling the user to start its own project from an existing one.

Figure 7.1 shows the ruby forge front page for the Ruby on Rails DB2 module.

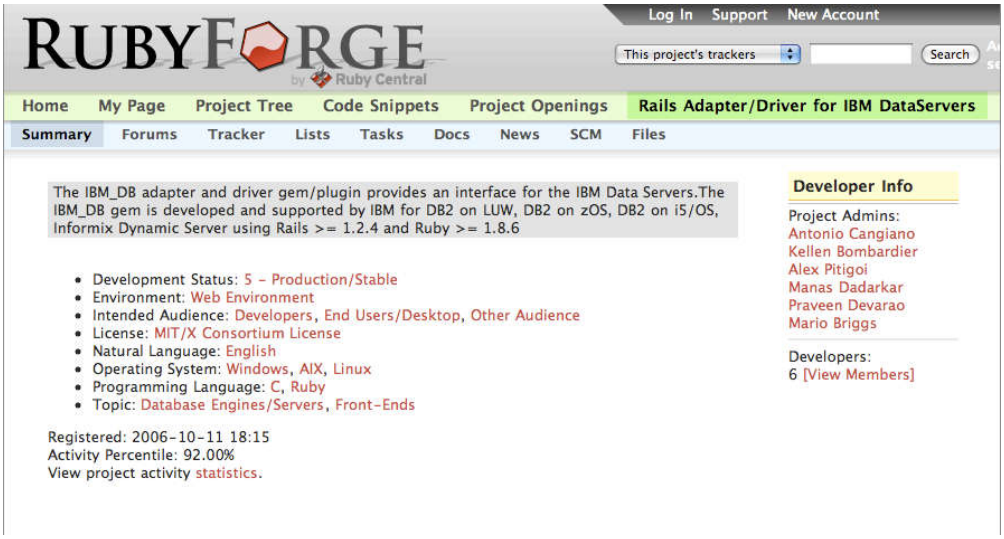


Figure 7.1 - The RubyForge site showing the DB2 - RoR module main page (top)

This is the top section of the DB2 - RoR module page, which consists of a quick project description, a list of the project developers and some menus on the horizontal bar. These menus give you direct access to different sections of the Web page, containing the tools we have talked through the book. The main menu bar includes the following (listed from left to right):

- **Summary**: Provides a summary of the project. This is what you can see in *Figure 7.1*
- **Forums**: A place where users can exchange their ideas.
- **Tracker**: Takes you to the bug reporting tool with several but similar options.
- **Lists**: Provides a listing of several mailing lists available with capability to view, subscribe and unsubscribe.
- **Tasks**: Provides a todo list that project developers use to organize their activities.
- **Docs**: For documentation
- **News** and the **SCM** area: Gives you instructions to check out the project through subversion
- **Files**: Takes you to the download area containing all different versions of the Rails DB2 driver package in different formats.

Figure 7.2 shows the bottom part of the ruby forge DB2 - RoR module page.

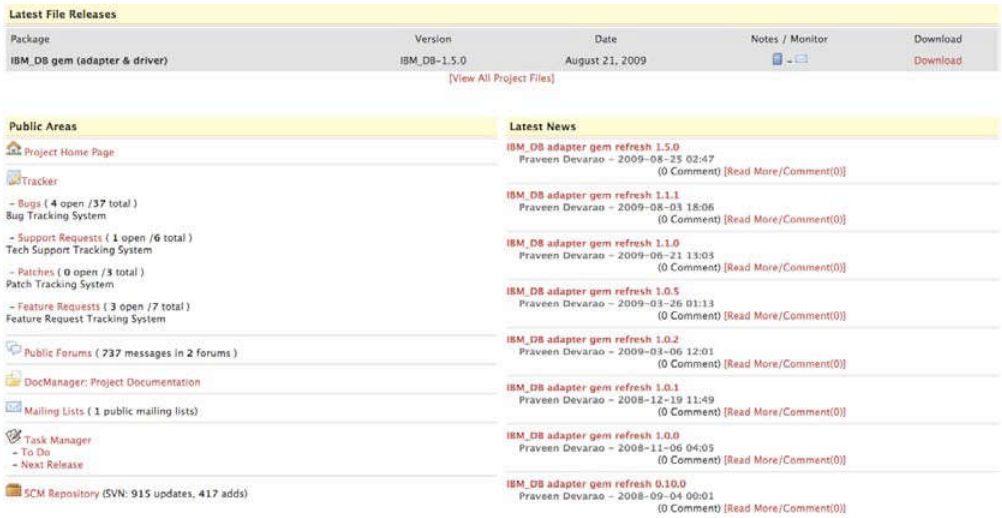


Figure 7.2 - The RubyForge site showing the DB2 - RoR module main page (bottom)

The bottom part of the Web page has a more detailed view of the items in the main menu, for example, the Latest File Releases area, gives you a description of the latest version as well as the date it came out and a quick link for downloading it. Below, the “Public Areas” section and the “Latest News” section follow the same principle. Either using the menu or these sections will lead you to the same content. It is important though that when you first arrive to a system like this you take some time to understand it. The sheer quantity of information with unusual nomenclatures can have an overwhelming impact.

7.3 Submitting a bug

The purpose of this chapter is to effectively use the knowledge we gathered from the rest of the book. One way of doing so is to imagine a situation where you have to submit a bug in the DB2 rails driver. As you have seen in the previous chapters, there are many ways of contributing to open source without having to type in a line of code. Submitting a bug may come across as trivial, but in fact most open source developers deal with poorly submitted bugs regularly.

Bugs exist since the first piece of software was created. Some bugs are nastier than others and can make software unusable. Bug reporting intends to decrease the amount of bugs in an application, making life easier for all of the users. A good bug report consists of the following:

- A detailed description of the system in which the software is running
- The input the user was feeding to the application
- The error message
- The expected behavior

Bug reports must be objective and based on facts, guesses have no place here. Even if you have the best of intentions by providing these guesses, it is not uncommon for a developer to spend a lot more time looking for a bug in a place there isn’t one, so the best thing you can do is just to fill in the bug with as much information as you can, and leave the rest to the developers. They will prompt you for input if they need to do so.

Let’s imagine for a moment that you find a real bug and you intend to fill in a bug report on the DB2 rails driver ruby forge page. "Where do I start?" you may ask. Well, the process below may help answer this question:

1. Check if there is already a submission for that bug. Duplicate bug submissions are a waste of time to you and to the developers.
2. If you don’t have an account on ruby forge, create one. If you have an account, login, then go to the ruby forge DB2 rails adapter page and select *tracker* from the main menu, then click on *Bugs*. Click *Submit New* button right below the main menu.
3. Fill in the fields:

- Category – This field requires some knowledge on the project structure. It refers to different sections of the project. For example the project may have a separate section for the user interface. In this case, the Ruby-DB2 project has two sections, Adapter and Driver. The adapter sits on rails ActiveRecord, which is a component responsible for the Object Relational Mapping (ORM). The driver, on the other hand, is responsible for making rails talk to IBM DB2 database.
- Group – The version of the IBM DB2 adapter and driver you are running.
- Summary – A short description of the bug.
- Detailed Description – This should include all the information necessary for the developer to reproduce the bug. In a system running rails, it would be obvious to start with the rails version, followed by the DB2 adapter and driver versions as well. All the error messages printed or logged by rails should be included. If you were doing anything at the time of the crash, describe it in detail.
- Upload Files – If you have a patch, attach and upload it. Sometimes when the output of the log files takes up too much space on the screen, it is nice to store it in one or more files and upload.

4. Press Submit and off you go.

Submitting bugs is simple science and one the best ways to contribute to an open source project; however, most people provide poor bug descriptions due to lack of knowledge or proper documentation. In this chapter you have seen how to fill a bug report in a simple and effective manner. There will be times when despite all your efforts, you won't make yourself clear. Be persistent and try to get your thoughts through clearly. The developers will appreciate it and you will have your software fixed sooner. Good Bug Hunting!